BIT.MLMR

Knowledge and Reasoning



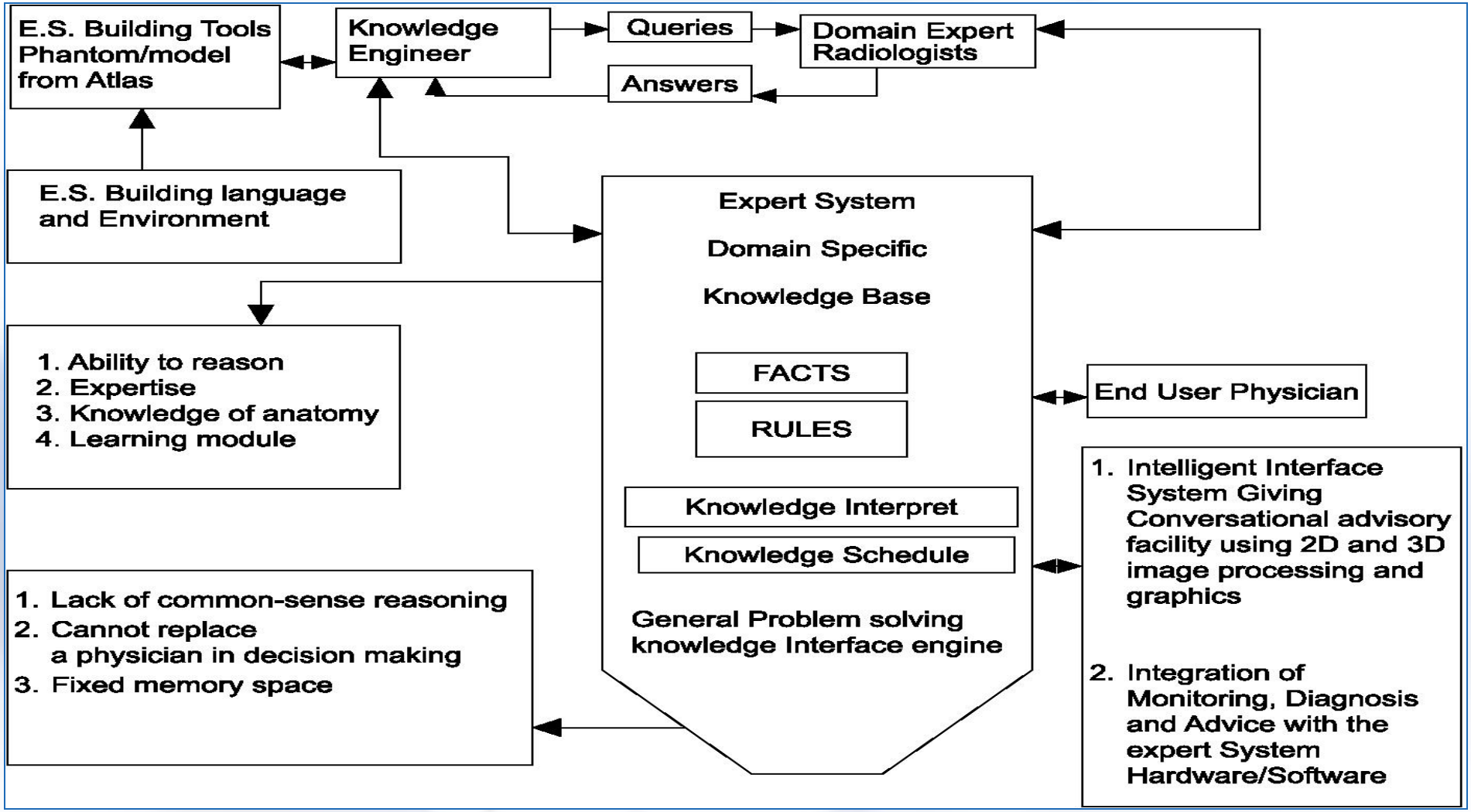
Lecture 5





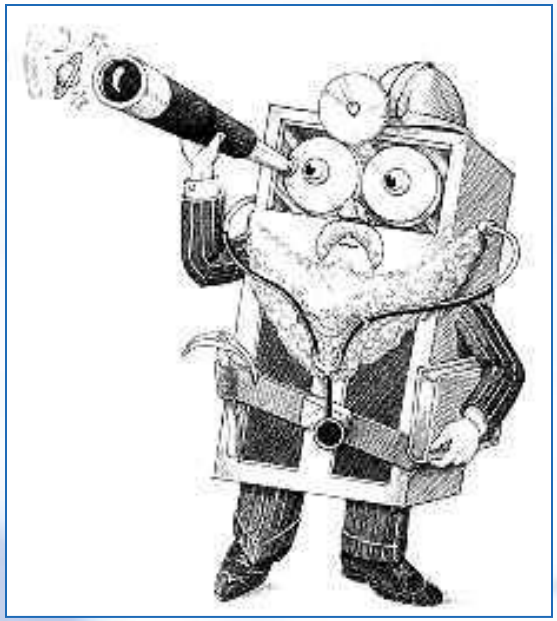


Structure of Expert Systems



From: www.emeraldinsight.com

An important rule based ES



MYCIN: HAS THE PATIENT RECENTLY HAD SYMPTOMS OF PERSISTENT HEADACHE OR OTHER ABNORMAL NEUROLOGIC SYMPTOMS (DIZZINESS, LETHARGY, ETC.)?
HUMAN: YES.
MYCIN: HAS THE PATIENT RECENTLY HAD OBJECTIVE EVIDENCE ABNORMAL NEUROLOGIC SIGNS (MUSCLE RIGIDITY, COMA, SEIZURES, ETC.) DOCUMENTED BY PHYSICIAN OBSERVATION OR EXAMINATION?
MYCIN: THE CSF CULTURES WILL BE CONSIDERED TO BE ASSOCIATED WITH MENINGITIS. PLEASE GIVE THE DATE ON WHICH CLINICAL EVIDENCE (SYMPTOMS, SIGNS OR LABORATORY TESTS) OF MENINGITIS FIRST APPEARED. PLEASE ENTER CSF FINDING. IS MENINGITIS A HOSPITAL-ACQUIRED INFECTION?
HUMAN: WHY?
MYCIN: THIS WILL AID IN DETERMINING THE ORGANISMS (OTHER THAN THOSE SEEN ON CULTURES OR SMEARS) WHICH MIGHT BE CAUSING THE INFECTION. IT HAS ALREADY BEEN ESTABLISHED THAT
……

A typical MYCIN rule:

IF

the type of x is primary bacteremia
the suspected entry point of x is the gastrointestinal tract
the site of the culture of x is one of the sterile sites

Then

there is evidence (0.8) that x is bacteroides

From:
http://www.atariarchives.org/deli/expert_systems.php

Sherlock Holmes



What is reasoning ?

- **Ability and Process of making decision based on facts and knowledge.**
  - ***Mechanism***

    How to do reasoning theoretically?
  - ***Control Strategy***

    How to realize the mechanism?

Mechanisms of Reasoning

- According to logicl basis

  Deduction, Induction, and Default

- According to the certainty of knowledge

  Reasoning under certainty vs. uncertainty

- According to the monotony of reasoning process

  Monotonic vs. Non-monotonic Reasoning

Control Strategies in Reasoning

- **Inference Direction**
  - Facts → Conclusions (Forward chain, Data-driven)
  - Facts ← Conclusions (Backward chain, Goal-driven)
  - Facts ↔ Conclusions (Bi-directional)
- **Conflict Resolution**
  - Sort knowledge for improving reasoning efficiency
- **Search**

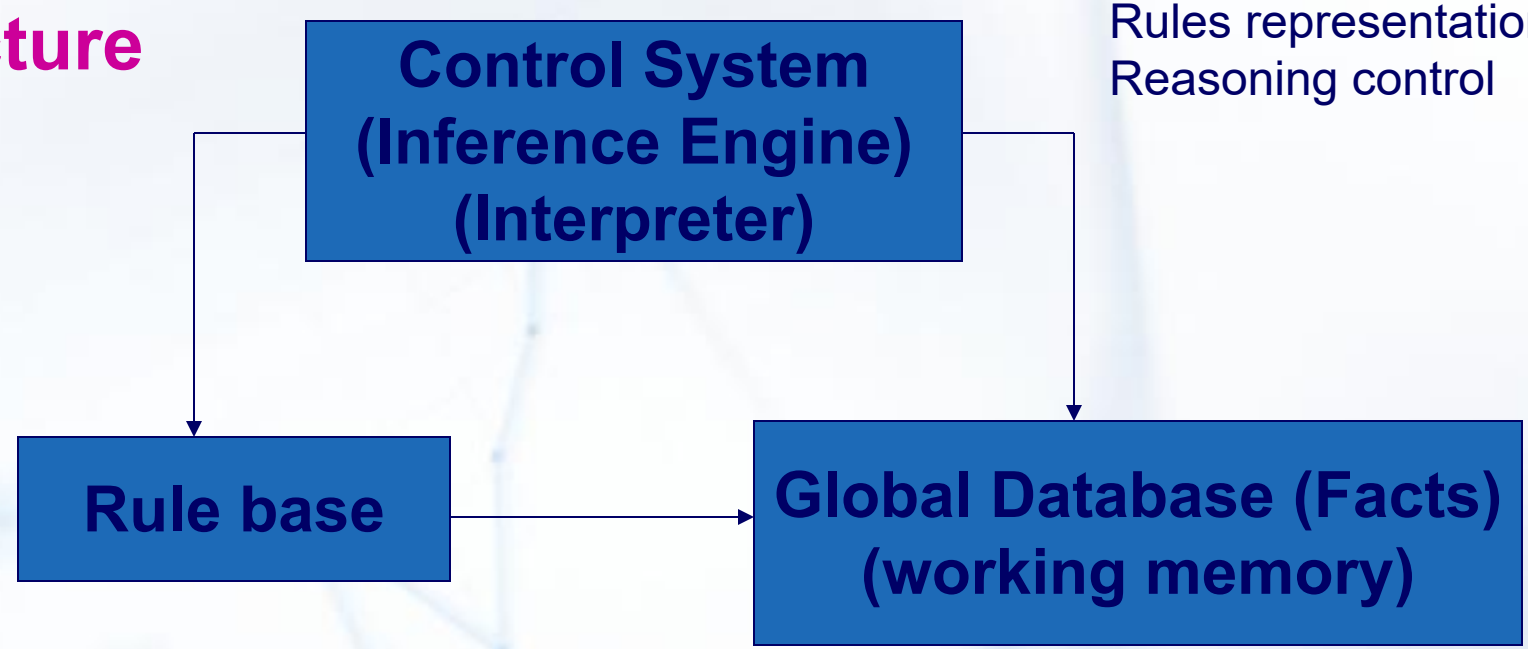
Outline

- **Production rules and system**
- **Inference in *First-order logic***
  - **Propositional logic**
  - **First-order logic**
  - ***Resolution***
- **Knowledge Graphs**

Part Ⅰ: Production Rule and System

Production-rule System

- **Structure**



Facts representation
Rules representation
Reasoning control

- **Procedure**

Rule Matching → Conflict Resolution → Rule Execution

Stop Conditions

Facts

- **Proposition**

  E.g., Snow is white. John likes AI.

- **Representation in computer:**

  Ternary:

  (attribute, object, value), (relation, object1, object2)

  E.g., (color, snow, white) (like, John, AI)

  Quaternion:

  (attribute, object, value, certification factor), (relation, object1, object2, certification factor)

Production Rule

P → Q

or IF P THEN Q (certainty factor)

Antecedent
Condition

Consequent
Conclusion or Action

E.g.,

IF *An animal has teeth, claw, and eyes staring into the frontage*

Then *the animal is carnivore* (0.6)

Production System Example 1

- **You want a program that can answer questions and make inferences about food items**
- **Like:**
  - What is purple and perishable?
  - What is packed in small containers?
  - What is green and weighs 5 kg?

Example 1: System Construction

WORKING MEMORY (WM)

Initially WM = (green, weighs-5-kg)

RULE BASE

R1. **IF** green **THEN** produce

R2. **IF** packed-in-small-container **THEN** delicacy

R3. **IF** [refrigerated **OR** produce] **THEN** perishable

R4. **IF** [weighs-5-kg **AND** inexpensive **AND NOT** perishable] **THEN** staple

R5. **IF** [weighs-5-kg **AND** produce] **THEN** watermelon

INTERPRETER

1. Find all productions whose condition parts are true
2. Deactivate productions that would add a duplicate symbol
3. Execute the lowest numbered production (or quit)
4. Repeat until there is no rule to execute

Example 1: First cycle of execution

WORKING MEMORY

WM = (green, weighs-5-kg)

CYCLE 1

1. Productions whose condition parts are true: ***R1***
2. No production would add duplicate symbol
3. Execute ***R1***.

   This gives: WM = (***produce,*** green, weighs-5-kg)

RULE BASE

R1. IF green THEN produce

R2. IF packed-in-small-container THEN delicacy

R3. IF [refrigerated OR produce] THEN perishable

R4. IF [weighs-5-kg AND inexpensive AND NOT perishable] THEN staple

R5. IF [weighs-5-kg AND produce] THEN watermelon

INTERPRETER

1. Find all productions whose condition parts are true
2. Deactivate productions that would add a duplicate symbol
3. Execute the lowest numbered production (or quit)
4. Repeat

Example 1: Second cycle of execution

WORKING MEMORY

WM = (produce, green, weighs-5-kg)

CYCLE 2

1. Productions whose condition parts are true: ***R1, R3, R5***
2. Production R1 would add duplicate symbol, so ***deactivate R1***
3. Execute ***R3*** because it is the lowest numbered production.

   This gives: WM = (***perishable,*** produce, green, weighs-5-kg)

RULE BASE

R1. IF green THEN produce

R2. IF packed-in-small-container THEN delicacy

R3. IF [refrigerated OR produce] THEN perishable

R4. IF [weighs-5-kg AND inexpensive AND NOT perishable] THEN staple

R5. IF [weighs-5-kg AND produce] THEN watermelon

INTERPRETER

1. Find all productions whose condition parts are true
2. Deactivate productions that would add a duplicate symbol
3. Execute the lowest numbered production (or quit)
4. Repeat

Example 1: Third cycle of execution

WORKING MEMORY

WM = (perishable, produce, green, weighs-5-kg)

CYCLE 3

1. Productions whose condition parts are true: ***R1, R3, R5***
2. Productions ***R1, R3*** would add duplicate symbol, so deactivate them
3. Execute ***R5***.

This gives: WM = (***watermelon***, perishable, produce, green, weighs-5-kg)

RULE BASE

R1. IF green THEN produce

R2. IF packed-in-small-container THEN delicacy

R3. IF [refrigerated OR produce] THEN perishable

R4. IF [weighs-5-kg AND inexpensive AND NOT perishable] THEN staple

R5. IF [weighs-5-kg AND produce] THEN watermelon

INTERPRETER

1. Find all productions whose condition parts are true
2. Deactivate productions that would add a duplicate symbol
3. Execute the lowest numbered production (or quit)
4. Repeat

Example1: Fourth cycle of execution

WORKING MEMORY

WM = (watermelon, perishable, produce, green, weighs-5-kg)

CYCLE 4

1. Productions whose condition parts are true: ***R1, R3, R5***
2. Productions ***R1, R3, R5*** would add duplicate symbol, so ***deactivate them***
3. ***Quit.***

What are the conclusions?

RULE BASE

R1. IF green THEN produce

R2. IF packed-in-small-container THEN delicacy

R3. IF [refrigerated OR produce] THEN perishable

R4. IF [weighs-5-kg AND inexpensive AND NOT perishable] THEN staple

R5. IF [weighs-5-kg AND produce] THEN watermelon

INTERPRETER

1. Find all productions whose condition parts are true
2. Deactivate productions that would add a duplicate symbol
3. Execute the lowest numbered production (or quit)
4. Repeat

Production system reasoning Summary

Step1. put the facts into database

Step2. if there is no rules whose condition part can map to the facts then fail.

Step3. Choose a rule whose condition part can map to facts

Step4. perform the rule, and add a tag on the rule. If the result part is an median result, then add it into database

Step5. If the target is in the database then success; Else goto Step2.

Part Ⅱ：Inference in First-order logic

2.1 Logic in general

- **Logics are formal languages for representing information such that conclusions can be drawn, can be used as a tool for knowledge representation and reasoning**

  - Syntax defines the sentences in the language

  - Semantics define the "meaning" of sentences;

  i.e., define truth of a sentence in a world

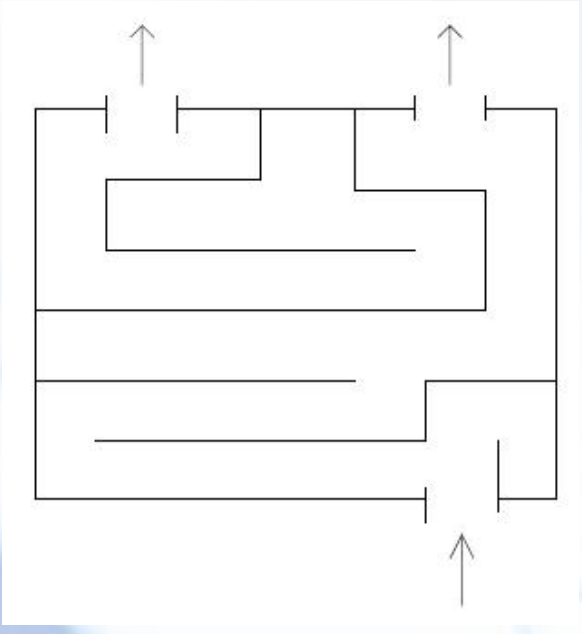
(1) Propositional logic

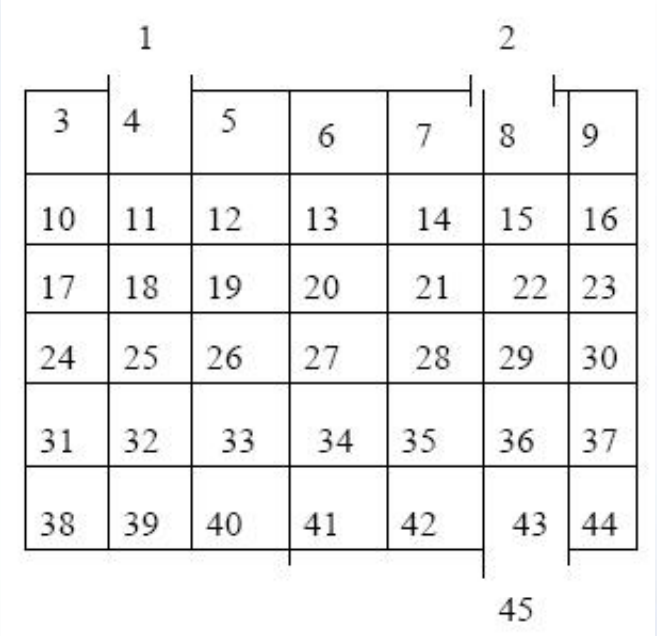
- Propositional logic is the simplest logic–illustrates basic ideas
- A proposition symbol represents an atomic sentence, Each proposition symbol can be specified true/false

E. g. P_1 : "3>5" false

P2 : "1+1=2" true

An example: Lybrinth





Fact: R45

Rules: R45->R43

R43->R36&R42

……

How to solve it through reasoning?

(2) First-order logic

- **Whereas propositional logic assumes the world contains facts**
- **first-order logic (like natural language) assumes the world contains**
  - Objects: people, houses, numbers, colors, baseball games, wars, …
  - Relations: red, round, prime, brother of, bigger than, part of, comes between, …
  - Functions: father of, best friend, one more than, plus, …

Syntax of FOL: Basic elements

- **Constants** E.g., John, 2, Sun,...
- **Functions** E.g., Sqrt, FatherOf,...
- **Variables** E.g., x, y, a, b,...

} Term

- **Predicates** E.g., Brother, >,...
- **Connectives** $\neg, \rightarrow, \wedge, \vee, \leftrightarrow$
- **Quantifiers** $\forall, \exists$

Atomic sentences

- **Atomic sentence = *predicate*($term_1$,...,$term_n$)**

- ***E.g.***
  1. *"Lingjian is a student": Student(lingjian)*
  2. *"Richard is the brother of John": Brother(Richard, John)*
  3. *"x>6": Greater(x, 6)*
  4. *"The father of Wanghong is a teacher": Teacher(father(Wanghong))*

Logic connectives

- **Complex sentences are constructed from atomic sentences using logic connectives**
  - If S is a sentence, $\neg S$ is a sentence (negation)
  - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (conjunction)
  - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (disjunction)
  - If S_1 and S_2 are sentences, $S_1 \rightarrow S_2$ is a sentence (implication)
  - If S_1 and S_2 are sentences, $S_1 \leftrightarrow S_2$ is a sentence (biconditional)

Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
$false$	$false$	$true$	$false$	$false$	$true$	$true$
$false$	$true$	$true$	$false$	$true$	$true$	$false$
$true$	$false$	$false$	$false$	$true$	$false$	$false$
$true$	$true$	$false$	$true$	$true$	$true$	$true$

Universal quantification

- ∀<*variables*> <*sentence*>

  E.g. "Everyone at BIT is smart":

  $\forall x\ At(x,BIT) \Rightarrow Smart(x)$

- **∀x *P* is true in a model *m* iff *P* is true with *x* being each possible object in the model**

- **Roughly speaking, equivalent to the conjunction of instantiations of *P***

  $At(John, BIT) \Rightarrow Smart(John)$
  $\wedge\ At(Richard, BIT) \Rightarrow Smart(Richard)$
  $\wedge\ At(Alice, BIT) \Rightarrow Smart(Alice)$
  $\wedge$ ...

Existential quantification

- ∃<*variables*> <*sentence*>

  E.g. "Someone at BIT is smart":
  ∃*x* At(x,BIT) ∧ Smart(x)

- **∃*x* *P* is true in a model *m* iff *P* is true with *x* being some possible object in the model**

- **Roughly speaking, equivalent to the disjunction of instantiations of *P***

  At(John, BIT) ∧ Smart(John)
  ∨ At(Richard, BIT) ∧ Smart(Richard)
  ∨ At(Alice, BIT) ∧ Smart(Alice)
  ∨ ...

Properties of quantifiers

- **$\forall x\ \forall y$ is the same as $\forall y\ \forall x$**

  $\exists x\ \exists y$ is the same as $\exists y\ \exists x$

- **$\exists x\ \forall y$ is not the same as $\forall y\ \exists x$**

  E.g., $\exists x\ \forall y$ Loves(x,y)

  - "There is a person who loves everyone in the world"

  but $\forall y\ \exists x$ Loves(x,y)

  - "Everyone in the world is loved by at least one person"

- **Quantifier duality: each can be expressed using the other**

  E.g.

  $\forall x$ Likes(x,IceCream) **$\neg\exists x\ \neg$Likes(x,IceCream)**

  $\exists x$ Likes(x,Broccoli) **$\neg\forall x\ \neg$Likes(x,Broccoli)**

Complex sentences

- **Complex sentences are made from atomic sentences using *Connectives* and *Quantifiers***

E.g.

1. **"Either Richard or John is the king":**

   king(Richard) ∨ king(John)

2. **"Every person has a father" :**

   (∀x)(∃y)(PERSON(x) ⇒ HASFATHER(x,y))

3. **"Every teacher has students" :**

(∀x)(∃y)(TEACHER(x) ⇒ TEACHES(x,y)∧STUDENT(y))

4. **"Every integer is either even or odd"**

   (∀x)(I(x) ⇒ E(x) ∨O(x))

2.2 Inference Foundation

- **Sentences are true with respect to a model and an interpretation**

  - Model contains objects (domain elements) and relations among them

  - Interpretation specifies referents for

constant symbols	→	objects
predicate symbols	→	relations {T, F}
function symbols	→	functional relations

An example of interpretation

- Predict: $A = (\forall x)(\exists y)P(x, y)$
- Domain: $D = \{1,2\}$

Interpretation 1:

P(1,1)	P(1,2)	P(2,1)	P(2,2)
T	F	T	F

Interpretation 2:

P(1,1)	P(1,2)	P(2,1)	P(2,2)
T	T	F	F

(1) Validity and Satisfiability

- A sentence is valid if it is true in all interpretations
- A sentence is satisfiable if it is true in some interpretations
- A sentence is unsatisfiable if it is true in no interpretations

(2) Equivalence and Entailment

- Two sentences are **logically equivalent** if they have same truth in all interpretations.
- A sentence P **entails** another sentence Q if and only if P→Q is valid.
- **Logical equivalence**, **entailment**, and other **rules** including **P rule**, **T rule**, **CP rule**, and **reduction to absurdity** are fundaments of inference in first-order logic

(3) Substitution and Unification

Substitution

$\{t_1/x_1, \cdots, t_n/x_n\}$ x_i is substituted by t_i

{a / x, c / y, f(b) / z} √ {g(y)/x, f(x)/y} ×

Unification

To introduce substitutions different logical expressions have to be look identical

An example of Unification

- **We can get the inference immediately if we can find a substitution θ such that *King(x)* and *Greedy(x)* match *King(John)* and *Greedy(y)***

  θ = {John/x, John/y} works

- **Unify(α,β) = θ if αθ = βθ**

p	q	θ
Knows(John,x)	Knows(John,Jane)	
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

An example of Unification

- We can get the inference immediately if we can find a substitution θ such that *King(x)* and *Greedy(x)* match *King(John)* and *Greedy(y)*

  θ = {John/x, John/y} works

- Unify(α,β) = θ if αθ = βθ

p	q	θ
Knows(John,x)	Knows(John,Jane)	{Jane/x}
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

An example of Unification

- **We can get the inference immediately if we can find a substitution θ such that *King(x)* and *Greedy(x)* match *King(John)* and *Greedy(y)***

  θ = {John/x, John/y} works

- **Unify(α,β) = θ if αθ = βθ**

p	q	θ
Knows(John,x)	Knows(John,Jane)	{Jane/x}
Knows(John,x)	Knows(y,OJ)	{OJ/x, John/y}
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

An example of Unification

- **We can get the inference immediately if we can find a substitution θ such that *King(x)* and *Greedy(x)* match *King(John)* and *Greedy(y)***

  θ = {x/John,y/John} works

- **Unify(α,β) = θ if αθ = βθ**

p	q	θ
Knows(John,x)	Knows(John,Jane)	{Jane/x}
Knows(John,x)	Knows(y,OJ)	{OJ/x, John/y}
Knows(John,x)	Knows(y,Mother(y))	{John/y, Mother(John)/x}
Knows(John,x)	Knows(x,OJ)	

An example of Unification

- We can get the inference immediately if we can find a substitution θ such that *King(x)* and *Greedy(x)* match *King(John)* and *Greedy(y)*

  θ = {x/John,y/John} works

- Unify(α,β) = θ if αθ = βθ

p	q	θ
Knows(John,x)	Knows(John,Jane)	{Jane/x}
Knows(John,x)	Knows(y,OJ)	{OJ/x, John/y}
Knows(John,x)	Knows(y,Mother(y))	{John/y, Mother(John)/x}
Knows(John,x)	Knows(x,OJ)	{fail}

Most General Unifier

- **To unify *Knows(John,x)* and *Knows(y,z)*,**
  θ = {John/y, z/x} or θ = {John/y, John/x, John/z}
- **The first unifier is more general than the second.**
- **There is a single most general unifier (MGU) that is unique up to renaming of variables.**
  MGU = {John/y, z/x}

MGU example

MGU for F={ P(a,x,f(g(y))) , P(z,h(z,u),f(u)) }

k=0: $F_0=F$; $\sigma_0=\varepsilon$

$D_0=\{a, z\}$, $\sigma_1=\sigma_0 \cdot\{a/z\}$

$F_1 = F_0(a/z)=$ { P(a, x, f(g(y))) , P(a, h(a, u), f(u)) }

k=1: $D_1=\{x, h(a,u)\}$, $\sigma_2=\sigma_1 \cdot\{h(a,u)/x\} =\{a/z, h(a,u)/x\}$

$F_2 = F_1(h(a,u)/x)=$ { P(a, h(a, u), f(g(y))) , P(a, h(a, u), f(u)) }

k=2: $D_2=\{g(y),u\}$, $\sigma_3=\sigma_2 \cdot\{g(y)/u\} =\{a/z, h(a,g(y))/x, g(y)/u\}$

$F_3 = F_2(g(y)/u)=$ { P(a, h(a, g(y)), f(g(y))), (P(a, h(a, g(y)), f(g(y))) }= { P(a, h(a, g(y)), f(g(y))) }

2.3 Resolution: Concepts

- **Literal**

  Atomic sentences and their negation

- **Clause**

  disjunction of literals

- **Empty clause (also □ or NIL)**

  unsatisfiable

- **Conjunctive Normal Form (CNF)**

  conjunction of clauses

Resolution: brief introduction

- A kind of **Reduction to absurdity**
- The problem of proving that P entail Q is converted as the problem of proving that

  $P \wedge \neg Q$ **is unsatisfiable.**

- Method: continue **resolving** two clauses until **empty clause** is derived

(1) Conversion to CNF

- **Example: Everyone who loves all animals is loved by someone:**

$\forall x\ ((\forall y(\ Animal(y) \rightarrow Loves(x,y))) \rightarrow (\exists y\ Loves(y,x)))$

1. Eliminate biconditionals and implications

$\forall x\ (\neg(\forall y(\ \neg Animal(y) \vee Loves(x,y))) \vee (\exists y\ Loves(y,x)]))$

2. Move $\neg$ inwards

$\forall x\ ((\exists y\ \neg(\neg Animal(y) \vee Loves(x,y))) \vee (\exists y\ Loves(y,x)))$

$\forall x\ ((\exists y(\neg\neg Animal(y) \wedge \neg Loves(x,y))) \vee (\exists y\ Loves(y,x)))$

$\forall x\ ((\exists y\ Animal(y) \wedge \neg Loves(x,y))) \vee (\exists y\ Loves(y,x)))$

Example contd.

3. Standardize variables: each quantifier should use a different one

$$\forall x\ ((\exists y\ Animal(y) \land \neg Loves(x,y))) \lor (\exists z\ Loves(z,x)))$$

4. Skolemize: a more general form of existential instantiation.

$$\forall x\ ((Animal(F(x)) \land \neg Loves(x,F(x))) \lor Loves(G(x),x))$$

5. Drop universal quantifiers:

$$(Animal(F(x)) \land \neg Loves(x,F(x))) \lor Loves(G(x),x)$$

6. Distribute $\lor$ over $\land$:

$$(Animal(F(x)) \lor Loves(G(x),x)) \land (\neg Loves(x,F(x)) \lor Loves(G(x),x))$$

(2) Resolution Rule

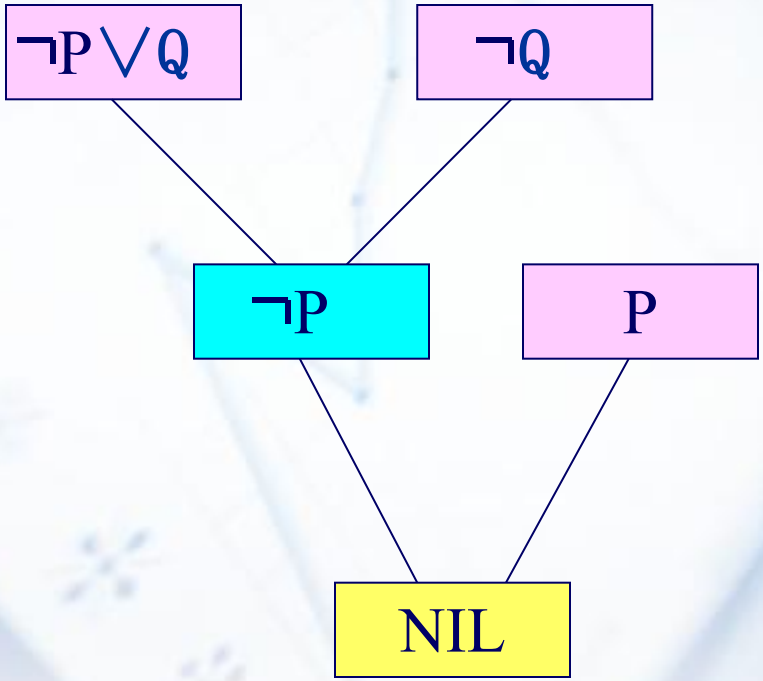
- **Eliminate complementary literals in two sentences, the disjunction of remaining parts of two sentences construct the resolvent clause which is the entailment of two sentences above.**

  Literals are complementary if one is the negation of the other.

- **Apply resolution steps to CNF(KB $\wedge \neg \alpha$); complete for inference when empty clause is reached.**

An example of resolution

- Let C1=¬P∨Q, C2= Q, C3=P, get the resolvent clause of C1, C2 and C3.



What is the meaning of NIL?

Proof of Resolution Rule

Resolution:

$$C_1 = L \vee C_1' \qquad C_2 = \neg L \vee C_2' \Rightarrow C_{12} = C_1' \vee C_2'$$

Proof:

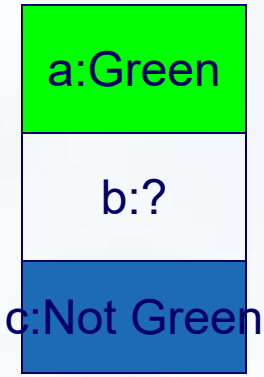
$$\because C_1' \vee L \Leftrightarrow \neg C_1' \to L \qquad \neg L \vee C_2' \Leftrightarrow L \to C_2'$$

$$\therefore C_1 \wedge C_2 = (\neg C_1' \to L) \wedge (L \to C_2')$$

$$(\neg C_1' \to L) \wedge (L \to C_2') \Rightarrow \neg C_1' \to C_2'$$

$$\because \neg C_1' \to C_2' \Leftrightarrow C_1' \vee C_2' = C_{12} \qquad \therefore C_1 \wedge C_2 \Rightarrow C_{12}$$

Example1 of FOL resolution proof



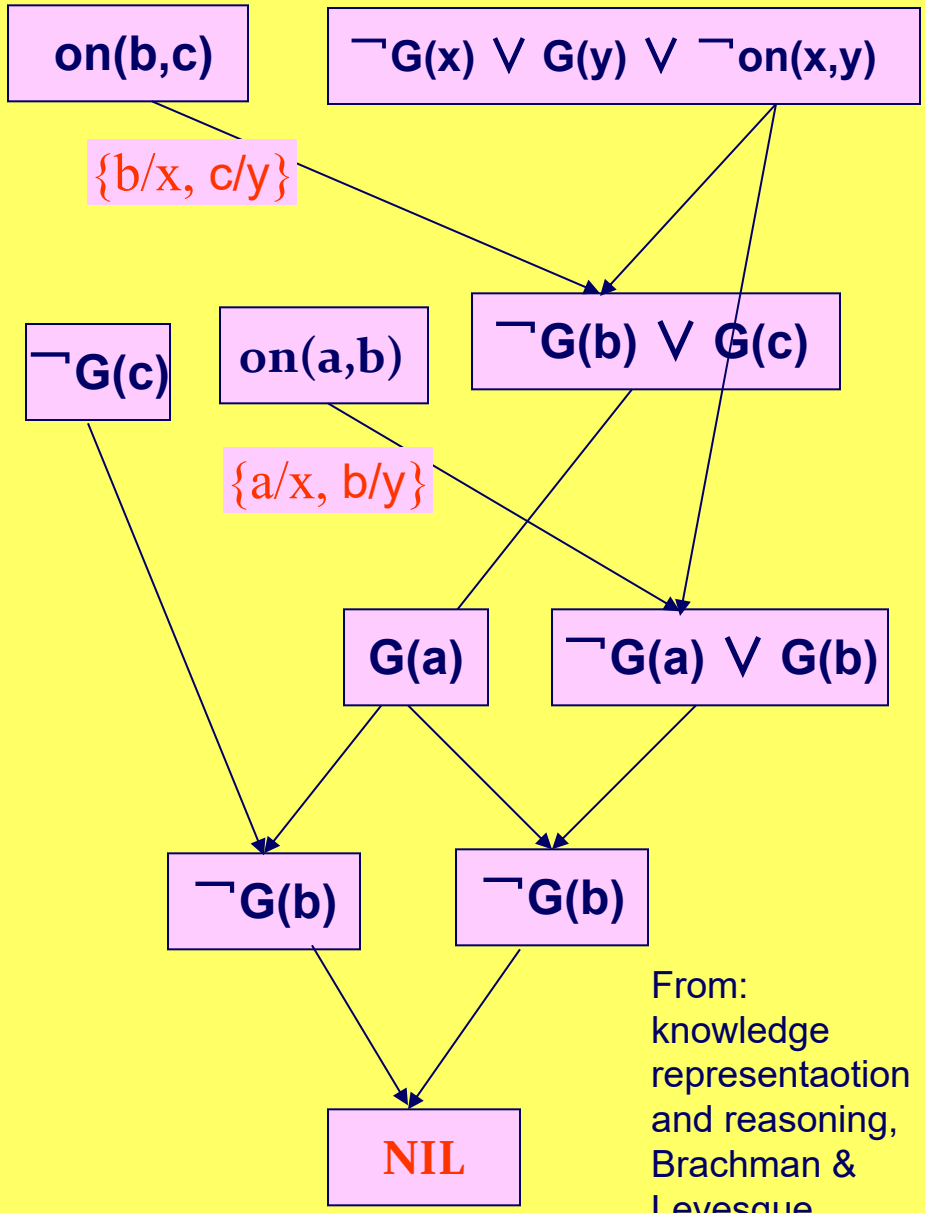
- Prove:there is a green block directly on top of a nongreen one.

- Facts:

$$on(a,b), on(b,c), G(a), \neg G(c)$$

- Negate Conclusion :

$$\neg(\exists x \exists y(G(x) \wedge \neg G(y) \wedge on(x,y)))$$
$$= \forall x \forall y(\neg G(x) \vee G(y) \vee \neg on(x,y))$$



From: knowledge representaotion and reasoning, Brachman & Levesque

Example2 of FOL resolution proof

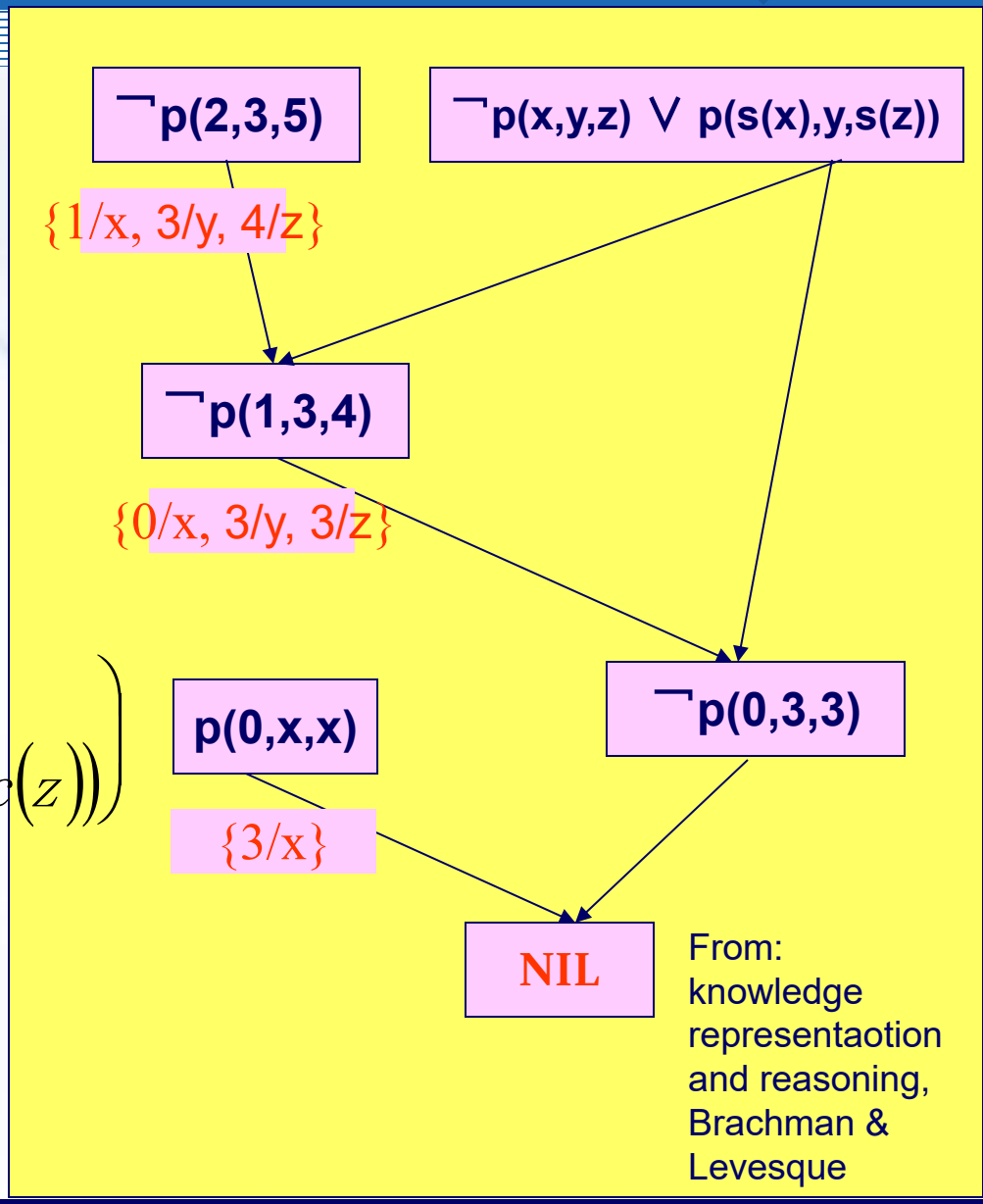
- Given addition properties:
  （1）0+x = x；（2）if x+y = z，then x+1+y=z+1
- Prove：2+3=5.
- Facts:

$$\forall x(plus(zero,x,x))$$

$$\forall x\forall y\forall z\left(\begin{array}{l} plus(x,y,z) \\ \rightarrow\ plus(succ(x),y,succ(z)) \end{array}\right)$$

- Negate Conclusion :

$$\neg plus(2,3,5)$$



Resolution Reasoning Example

- **Facts and Rules:**

  Jack owns a dog.

  Every dog owner is an animal lover.

  No animal lover kills an animal.

  Either Jack or Curiosity killed the cat, who is named Tuna.

- Problem:

  Did Curiosity kill the cat?

Resolution Reasoning Example

- Resolution Step1:

  Express the facts and Rules in FOL:

A:$(\exists x)\ Dog(x) \wedge Owns(Jack,x)$

B:$(\forall x)((\exists y)\ Dog(y) \wedge Owns(x, y)) \rightarrow AnimalLover(x)$

C:$(\forall x)(\forall y)\ AnimalLover(x) \wedge Animal(y) \rightarrow \neg Kills(x, y)$

D: $Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$

E: $Cat(Tuna)$

F: $\forall x\ Cat(x) \rightarrow Animal(x)$

Resolution Reasoning Example

- **Resolution Step2:**

  Conversion to CNF:

  A1: $Dog(D)$

  A2: $Owns(Jack, D)$

  B: $\neg Dog(f(x)) \vee \neg Owns(x, f(x)) \vee AnimalLover(x)$

  C: $\neg AnimalLover(x) \vee \neg Animal(y) \vee \neg Kills(x, y)$

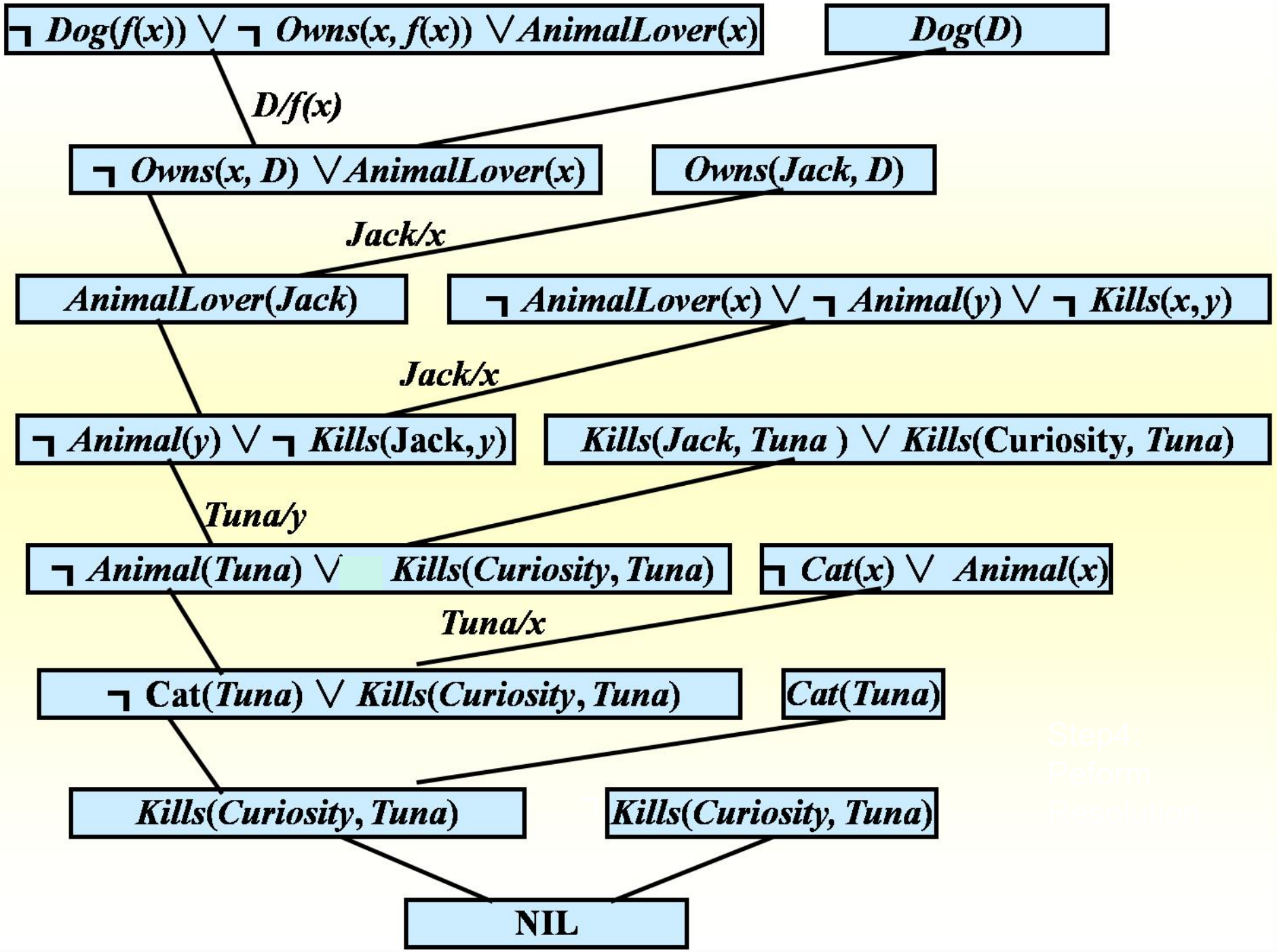
  D: $Kills(Jack, Tuna) \vee Kills(\text{Curiosity}, Tuna)$

  E: $Cat(Tuna)$

  F: $\neg Cat(x) \vee Animal(x)$

- Resolution Step3:

  negation of the result: $\neg Kills(Curiosity, Tuna)$


¬ Dog(f(x)) ∨ ¬ Owns(x, f(x)) ∨ AnimalLover(x)
Dog(D)
D/f(x)
¬ Owns(x, D) ∨ AnimalLover(x)
Owns(Jack, D)
Jack/x
AnimalLover(Jack)
¬ AnimalLover(x) ∨ ¬ Animal(y) ∨ ¬ Kills(x,y)
Jack/x
¬ Animal(y) ∨ ¬ Kills(Jack,y)
Kills(Jack, Tuna) ∨ Kills(Curiosity, Tuna)
Tuna/y
¬ Animal(Tuna) ∨ Kills(Curiosity, Tuna)
¬ Cat(x) ∨ Animal(x)
Tuna/x
¬ Cat(Tuna) ∨ Kills(Curiosity, Tuna)
Cat(Tuna)
Kills(Curiosity, Tuna)
Kills(Curiosity, Tuna)
NIL

Summary of Resolution in prove

Step1. Converse the facts and rules into clause form - a set of clauses set

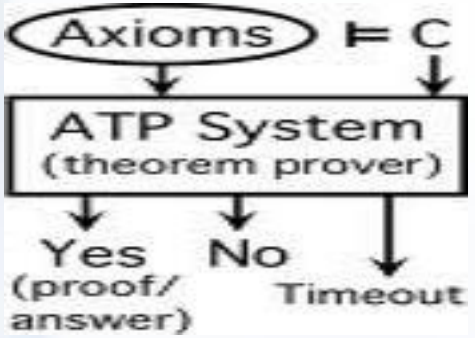
Step2. Add the negation of what is to be proved, in clause form, together to the clause set

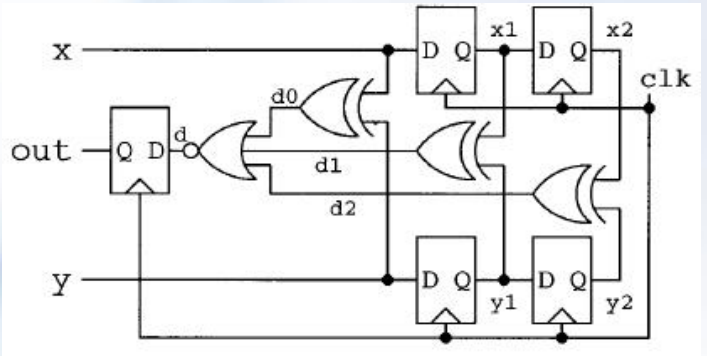
Step3. Resolve these clauses together, producing new clauses the logically follow from them

Step4. Iteratively apply resolution to the clauses and add results to either until there no more resolvent that can added or until the empty clause is produced.

Applications of FOL

- Automatic Threorem Proving
- Semantic Web
- Conecptural Clustering
- Reason about policies
- Formal Verification in hardware design
- Game and Automata
- ……..



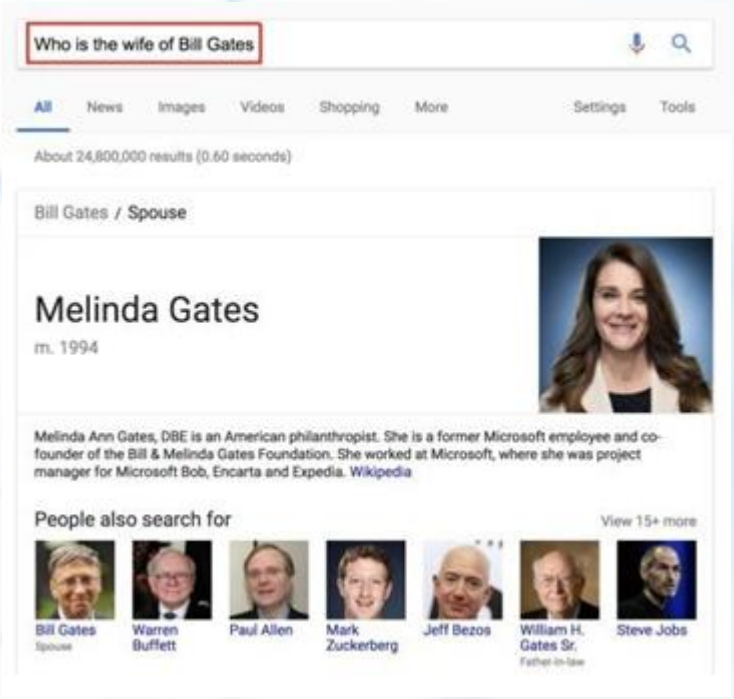


Part Ⅲ: Knowledge Graph

3.1. What is Knowledge Graph

Knowledge Graph

- **Google announced its Knowledge Graph on May 16, 2012, as a way to significantly enhance the value of information returned by Google searches.**



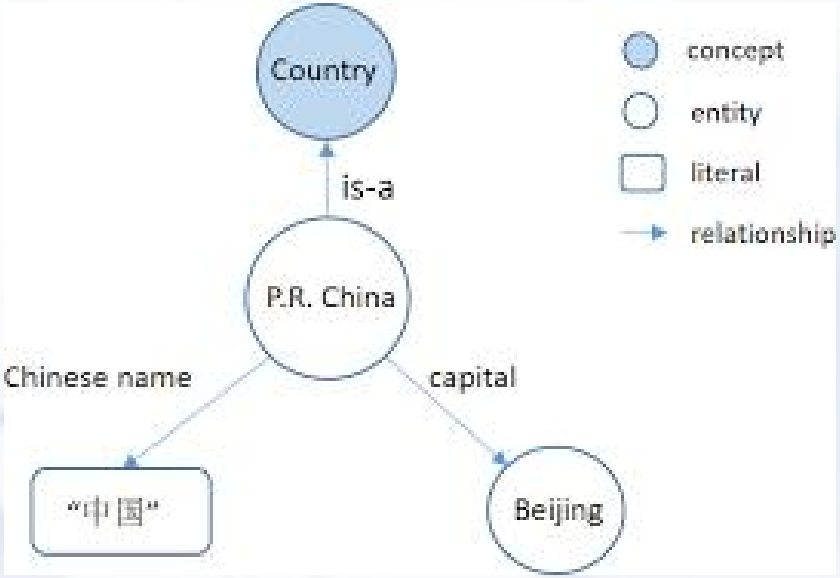
Knowledge Graph

- **What is a Knowledge Graph?**

- **A knowledge graph, also known as a semantic network, represents a network of real-world entities—i.e. objects, events, situations, or concepts—and illustrates the relationship between them.**

Knowledge Graphs Definition

- A knowledge graph is a directed labeled graph in which the labels have well-defined meanings.
- A directed labeled graph consists of nodes, edges, and labels.

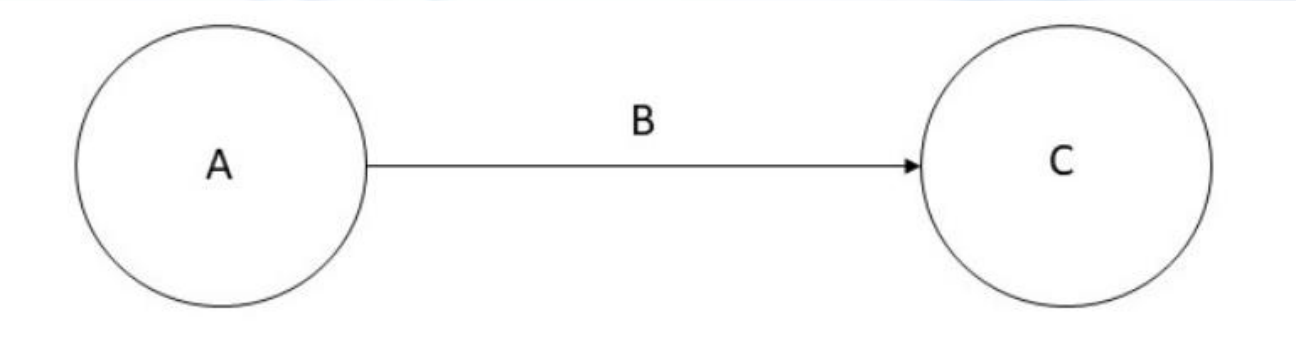


Knowledge Graphs Definition

- Anything can act as a node, for example, people, company, computer, etc.
- An edge connects a pair of nodes and captures the relationship of interest between them, for example, friendship relationship between two people, customer relationship between a company and person, or a network connection between two computers.
- The labels capture the meaning of the relationship, for example, the friendship relationship between two people.

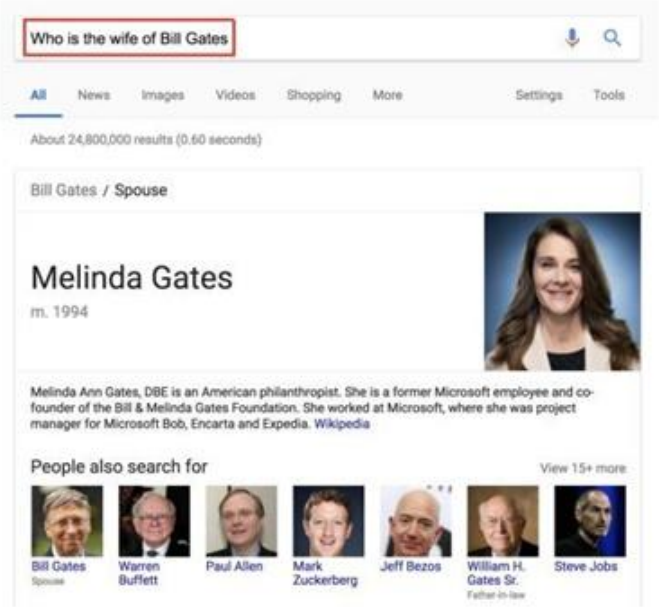
Knowledge Graphs Definition

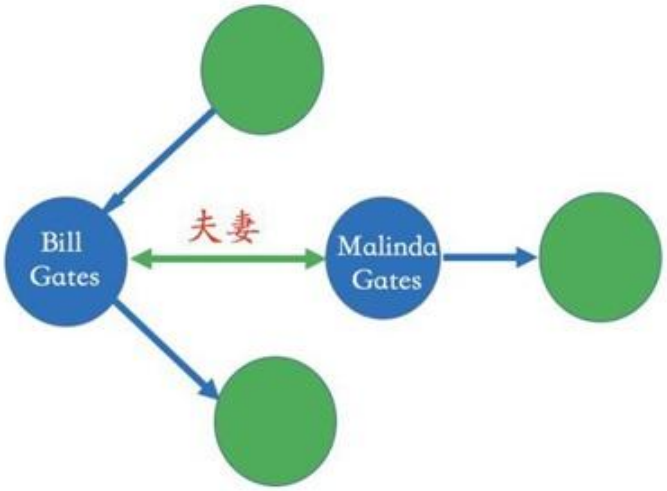
- given a set of nodes *N*, and a set of labels *L*, a knowledge graph is a subset of the cross product $N \times L \times N$. Each member of this set is referred to as a **triple**, and can be visualized as shown below.



Knowledge Graph

- E.g. “Who is the wife of Bill Gates?”





3.2. Knowledge Graph Construction

Knowledge Graph

- Create a Knowledge Graph from Data?
- Schema design
  - Relating the schema of sources to the knowledge graph schema
- Record linkage
  - Recognizing if two instances refer to the same object in the real-world

Knowledge Graph Construction

- Create a Knowledge Graph from Unstructured Input? (Hard)
- 4 Steps:
  - 1. Entity Extraction
  - 2. Relation Extraction
  - 3. Entity Resolution
  - 4. Coreference Resolution

Knowledge Graph Construction

❖ Entity Extraction

❖ Consider the following piece of text:

Cecilia Love, 52, a retired police investigator who lives in Massachusetts, said she paid around $370 a ticket with tax for nonstop United Airlines flights to Sacramento from Boston for her niece's high school graduation in June, 2020.

Knowledge Graph Construction

- A named entity is anything that can be referred to with a proper name: a person, a location, an organization, etc.
- The definition of a named entity is commonly extended to include things that are not entities per se, including dates, times, and other kinds of expressions involving time, and even numerical expressions, for example, prices.

[PER Cecilia Love], 52, a retired police investigator who lives in [LOC New Jersey], said she paid around [MONEY $370] a ticket with tax for nonstop [ORG United Airlines] flight to [LOC Sacramento] from [LOC Boston] for her niece's high school graduation in [TIME June, 2020].

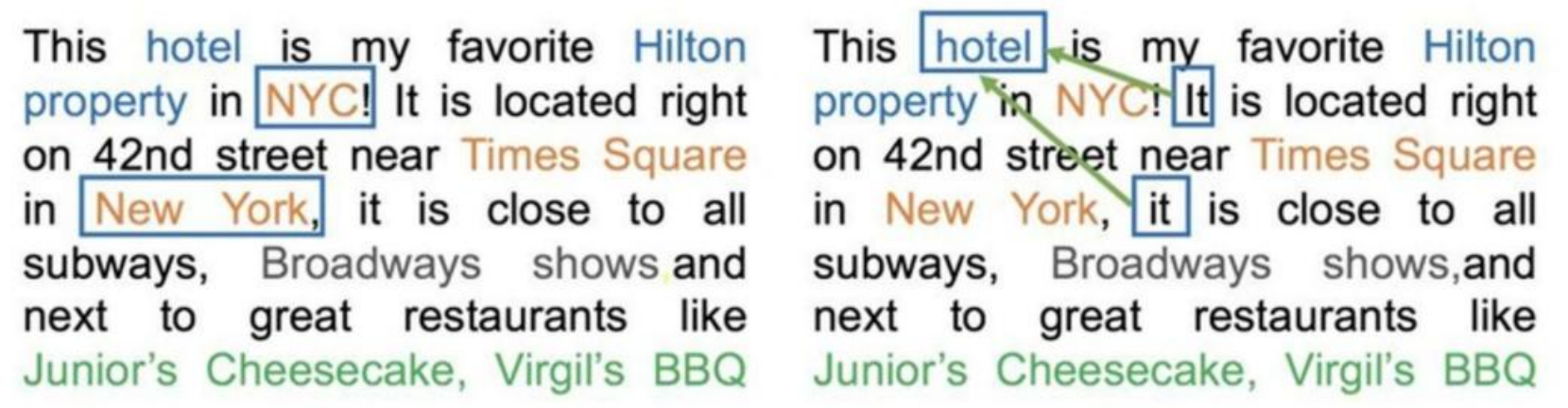
Knowledge Graph Construction

Relation Extraction

- There are three broad approaches to relation extraction:
  - syntactic patterns
  - various forms of supervised machine learning
  - unsupervised machine learning.
- The unsupervised machine learning has limited use in practice.

Knowledge Graph Construction

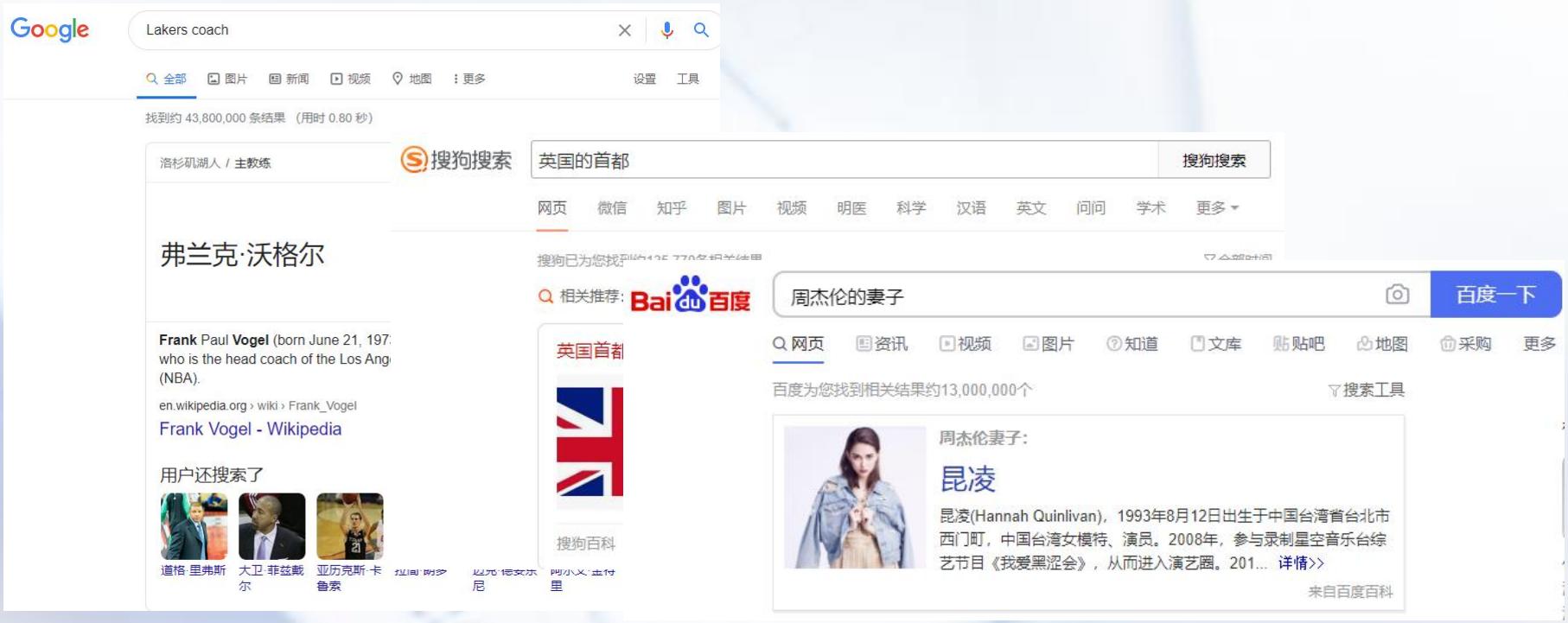
- **Entity Resolution & Coreference Resolution**



3.3. Typical Application of Knowledge Graph

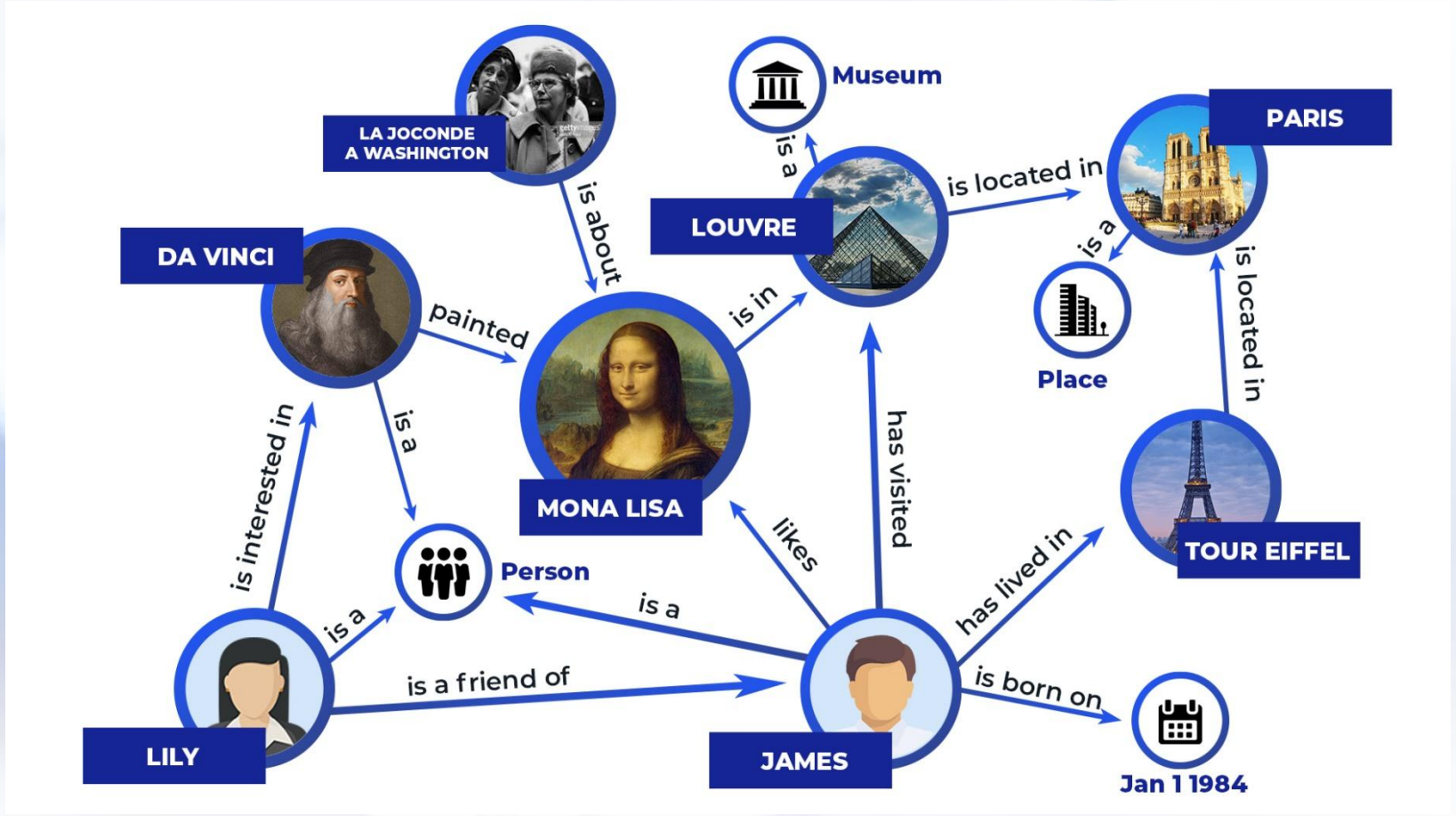
Knowledge Graph Application

❖ 1. Search engines



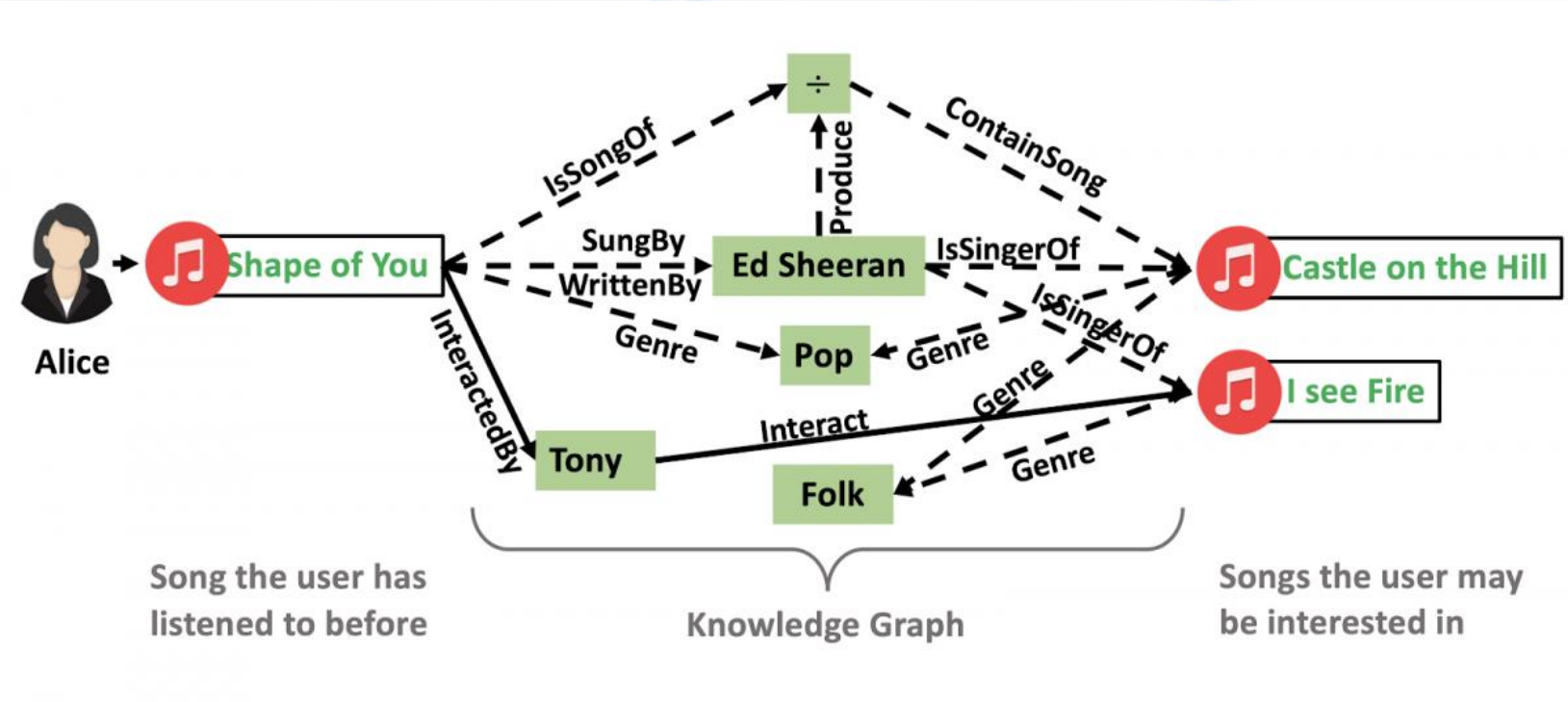
Knowledge Graph Application

❖ 2. Question Answering



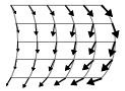
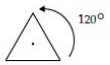
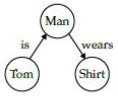
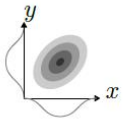
Knowledge Graph Application

❖ 3. Recommendation Engine



Knowledge Graphs enhanced Learning

How to Think?

Algebraic Equations	Differential Equations	Simulation Results	Spatial Invariances	Logic Rules	Knowledge Graphs	Probabilistic Relations	Human Feedback
$E = m \cdot c^2$ $v \leqslant c$	$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$ $F(x) = m\frac{d^2 x}{dt^2}$			$A \wedge B \Rightarrow C$			

- Hypothesis Set
- Objectiveness
- Optimization
- Training Data

Key Elements in Learning

Approach: Integration into the Hypothesis Set

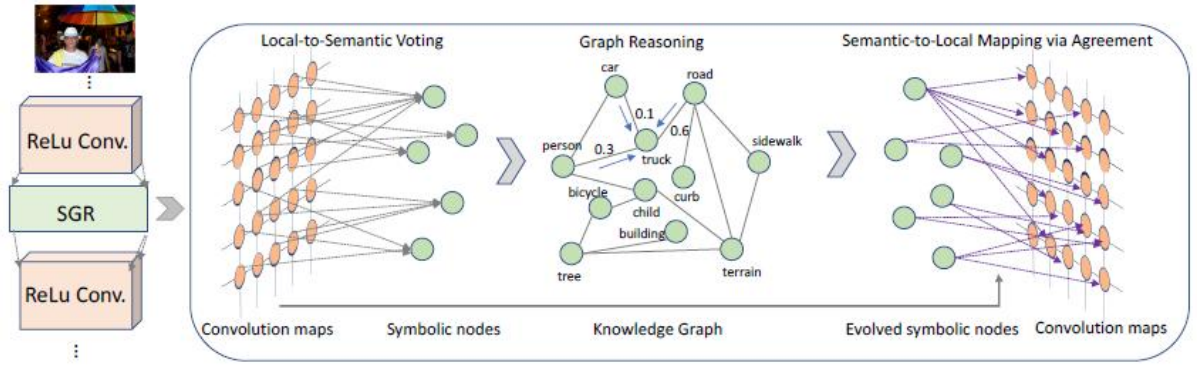
- Inference about a particular object is facilitated by using relations to other objects in an image.
- Graph neural networks operate on graphs and thus feature an object-and relation-centric bias in their architecture.
- A graph reasoning layer can be inserted into any neural network: Enhance representations in a given layer by propagating through a given knowledge graph.

Symbolic Graph Reasoning Meets Convolutions

- How to harness various external human knowledge for endowing the networks with the capability of semantic global reasoning?
- Propose a new Symbolic Graph Reasoning (SGR) layer, which performs reasoning over a group of symbolic nodes whose outputs explicitly represent different properties of each semantic in a prior knowledge graph.
- The SGR layer can be injected between any convolution layers and instantiated with distinct prior graphs.

Liang et al., "Symbolic graph reasoning meets convolutions," in Neural Information Processing Systems (NIPS), 2018.

Symbolic Graph Reasoning



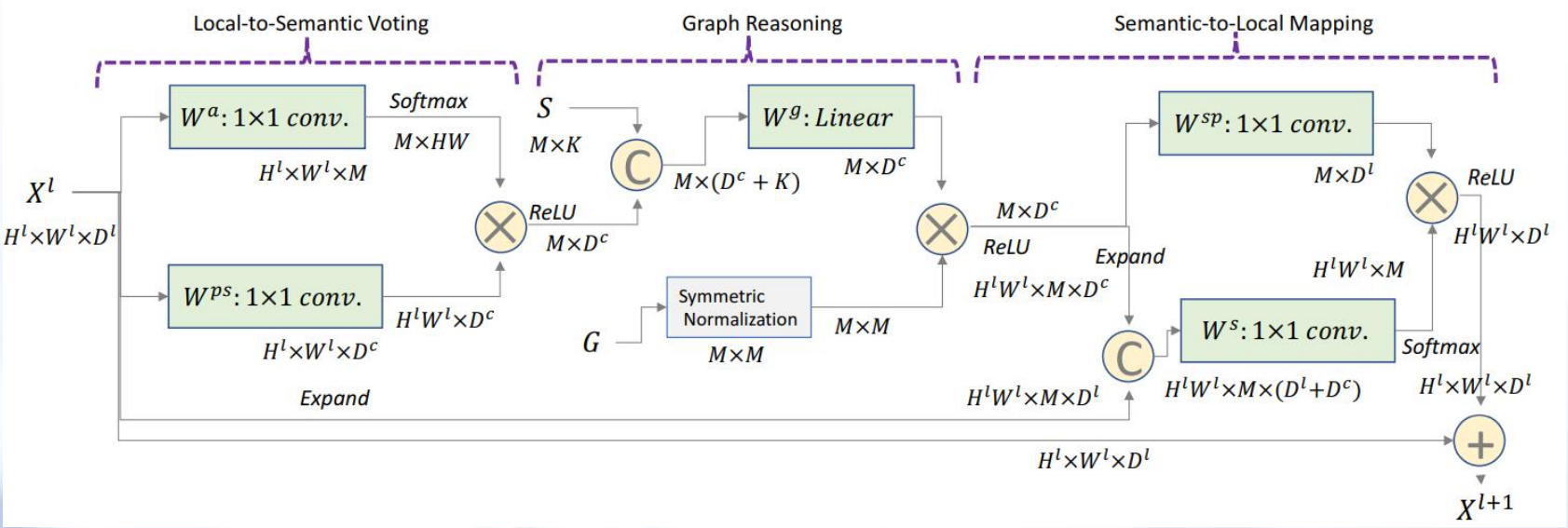
- To leverage global graph reasoning to enhance local features with external structured knowledge
- Consists of three modules:

  ① local-to-semantic voting: local features that are correlated to a specific semantic meaning (e.g. cat) are aggregated to depict the characteristic of its corresponding symbolic node.

  ② graph reasoning: leverage semantic constraints from human commonsense to evolve global representations of symbolic nodes.

  ③ semantic-to-local mapping: the evolved global representations of symbolic nodes can be used to further boost the capability of each local feature representation.

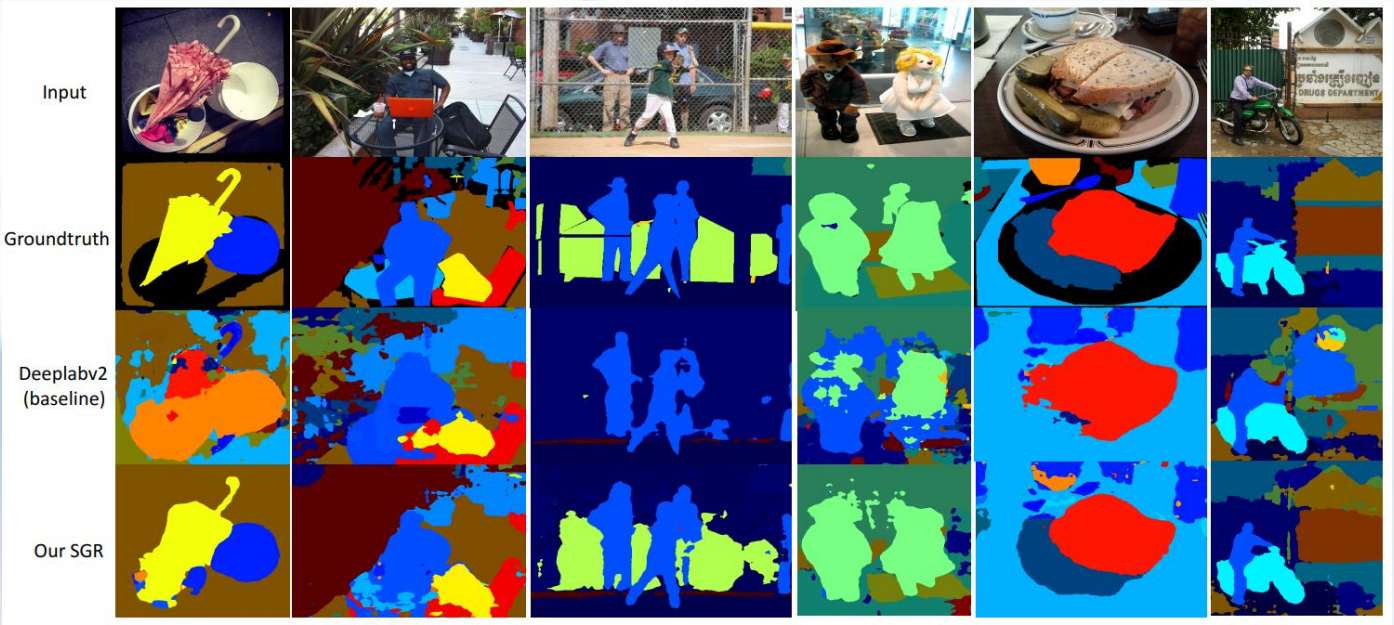
Symbolic Graph Reasoning



- The SGR layer is instantiated by specific knowledge graph with different numbers of symbolic nodes and distinct node connections.
- Combining multiple SGR layers with distinct knowledge graphs into convolutional networks can lead to hybrid graph reasoning behaviors.

Example: Image Segmentation

- Use a universal concept hierarchy: starting from the label hierarchy of COCO-Stuff that includes 182 concepts and 27 super-classes, we manually merge concepts from the rest two dataset together by using WordTree. It results in 340 concepts in the final concept graph.



BIT.MLMR

End of Lecture 5

Can you develop an intelligent software for reasoning with knowledge?

LIUXIABI@BIT.EDU.CN





